Requirements

Product Requirements

Elicitation. We began by reviewing the customer's product brief and the assessment paper. The brief gave an initial set of requirements but left several unclear or missing, particularly quality requirements such as performance, supported platforms and visual style. We prepared targeted questions for our first customer meeting, for example "What hardware should this run on, and does it need to support mobile devices?" and "Should the game be 2D or 3D, and which viewpoint would you prefer?" The customer specified that the game must run on Windows, macOS and Linux, should launch quickly to hold players' interest, and should use a top-down 2D view. They also clarified the game's purpose: introducing prospective students to the University of York and its well-known geese.

Negotiation. Several requirements were agreed through discussion rather than simply recorded. The choice of 2D over 3D balanced the customer's preference against what was achievable within the project timeline. We also proposed features the brief did not mention, such as saving progress, and checked with the customer before including them. Where the brief was silent on features essential to a working game, such as pausing and a fullscreen toggle, we added them as low-complexity requirements that do not conflict with any stated need. Open questions will be raised at future meetings, and requirements will be updated as the project develops.

Presentation. Following Sommerville [1], we separate *user requirements*, written in plain language so the customer can check them, from *system requirements*, which state precisely what the system must do so the team can build and test against them. System requirements are divided into functional requirements (FR_), describing what the game does, and quality requirements (QR_), describing how well it must do it [1]. As in the Volere template [2], constraints (CR_) are recorded separately because they are fixed conditions on the solution, such as platform and programming language, rather than features, and each records its source: the brief, the assessment paper or a client meeting.

Every requirement has a unique, descriptive identifier so it can be referenced in later deliverables, and a priority using the keywords defined in ISO/IEC/IEEE 29148 [3]: *Shall* for mandatory, *Should* for desirable and *May* for optional requirements, which can be dropped first if time runs short. Each system requirement links to the user requirement it satisfies, supporting the traceability the standard calls for [3], so we can confirm every customer need is covered and trace it through design, implementation and testing. Constraints and quality requirements include a measurable fit criterion [2], because statements such as "the game should start quickly" cannot be tested without one [1]. Presenting requirements in tables makes them easy to scan and compare, for both the customer and the team.

References

[1] I. Sommerville, *Software Engineering*, 10th ed. Harlow, UK: Pearson, 2016.
[2] S. Robertson and J. Robertson, *Mastering the Requirements Process: Getting Requirements Right*, 3rd ed. Upper Saddle River, NJ: Addison-Wesley, 2012.
[3] *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE 29148:2018, 2018.

Constraint Requirements :

ID	Description	User requirements	Fit criteria
CR_PLATFORM	The game shall run on Windows, macOS and linux desktops	Client meeting	A full playthrough completes on each OS with no crash
CR_HARDWARE	The game shall run on a standard desktop computer without specialised hardware	Client meeting	Full playthrough on the listed minimum spec (4 core cpu 8gb Ram integrated graphics)
CR_JAVA	The game shall be written in Java 17	Assessment paper	The build config targets Java version 17
CR_VIEW	The game should use a top down 2d view	Client meeting	Check if its 2d or 3d visually
CR_JAR	Game delivered on executable JAR	Assessment paper	Jar launches on minimum spec computer
CR_LICENCE	Third party assets and libraries should be properly licensed and attributed	brief	Every library and asset should be allowed for use in commercial settings
CR_PEACE	The game shall not depict geese being harmed	Client meeting	Manual review showing no death blood or injury to geese sprites

User Requirements:

(user requirements ID)	Description	Priority
UR_EVENTS	The user will experience events which can negatively or positively alter the user's experience	shall
UR_RELATE	The system will be familiar to university of york students	should
UR_GEESE	Geese will provide as an obstacle to the players completion	shall
UR_SCORE	The system should implement a scoring system which rewards good item management and timely progress	shall
UR_ITEMS	The user will have access to items which they can use to help them dissolve the geese	shall
UR_MOVEMENT	The user should be able to control their character	shall
UR_GAME_END	The game should end in a clear win or loss	shall
UR_ACCESSABILITY	The game should be accessible for players of all different skills and abilities	should

Functional Requirements:

ID	Description	User_requirement
FR_MOVE_KEYS	The character should move up/down/left/right using arrow and WASD keys	UR_MOVEMENT
FR_GEESE_BLOCK	Geese should block player movement	UR_GEESE
FR_GEESE_WAVES	Geese should approach the player in waves	UR_GEESE
FR_START_ITEMS	The player starts with 5 inventory items	UR_ITEMS
FR_MAP_ITEMS	At least 5 pickup items are placed on the map	UR_ITEMS
FR_ITEM_ICON	The inventory visually represents each item through little icons	UR_ITEMS
FR_ITEM DISPERSE	Using an item disperses nearby geese	UR_ITEMS, UR_GEESE
FR_SCORE_ITEMS	Score increases when an item disperses geese	UR_SCORE
FR_SCORE_DISPLAY	Faster dispersal gives a higher score	UR_SCORE
FR_TIMER	A 5 minute countdown is visible	UR_GAME_END
FR_WIN	Game ends in a win when geese are 100% dispersed	UR_GAME_END
FR_LOSE	LOSS screen appears if the timer ends and the win condition is unmet	UR_GAME_END
FR_EVENTS_COUNT	At least 5 triggerable events exist	UR_EVENTS
FR_EVENT_AUDIO	Each event should have a distinct sound	UR_ACCESSABILITY
FR_LANDMARKS	The map shall include at least 2 uni of york landmarks (central hall, SLB etc)	UR_RELATE
FR_CHAR_LOOK	The player character should be identifiable as a uni of york undergrad	UR_RELATE
FR_PAUSE	The player should be able to pause the game at any time	UR_ACCESSABILITY
FR_FULLSCREEN	The player should be able to toggle fullscreen mode	UR_ACCESSABILITY
FR_START_PAUSED	The game shall start paused	UR_ACCESSABILITY
FR_MAP_BOUNDS	The player cannot move outside the bounds of the map	UR_MOVEMENT
FR_USAGE_COUNTER	The system shall display the number of items used and events triggered	UR_EVENTS, UR_ITEMS

FR_PAUSE_MENU	Pausing will open up a pause menu with options to change settings	UR_ACCESSABILITY
FR_VOLUME	The game should have the option to lower or raise volume	UR_ACCESSABILITY

Quantity Requirements:

ID	Description	User requirements	Fit criteria
QR_STARTUP	The game should start quickly	UR_ACCESS	The main menu appears within 7 seconds of launching the JAR in 5 consecutive launches
QR_FRAME_RATE	The gameplay should run smoothly	UR_ACCESS	Frame rate stays at 30 fps for whole 5 min session with max geese on screen
QR_OPERABILITY	A first-time player should be able to understand the premise and play the game without external instruction	UR_ACCESS	Test players should be able to start the game, disperse geese use items and activate events with no external help
QR_WINNABILITY	Most players should be able to win the game	UR_ACCESS	70% of test players win within their first attempt
QR_HUD_VISABILITY	The time,score and inventory should remain readable in all display modes	UR_ACCESS	At 1280*72 and 1920*1080 in windowed and fullscreen no HUD element is overlapped